ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KĨ THUẬT MÁY TÍNH




# BÁO CÁO LAB 2

## Môn: Vi xử lý - vi điều khiển(TH)

**GVHD: Phan Văn Sỹ**

**Sinh viên thực hiện : Trần Thanh Hải**

**MSSV : 2410915**

**Học kỳ 261, tháng 9 năm 2026**

# Mục lục


**1 Exercise 1** **2**
- 1.1 Sơ đồ Proteus . . . 2
- 1.2 Code . . . 2
- 1.3 Short question . . . 3

**2 Exercise 2** **3**
- 2.1 Sơ đồ Proteus . . . 3
- 2.2 Code . . . 3
- 2.3 Short question . . . 4

**3 Exercise 3** **4**
- 3.1 Code . . . 4
- 3.2 Code . . . 5

**4 Exercise 4** **5**
- 4.1 Code . . . 5

**5 Exercise 5** **6**
- 5.1 Code . . . 6

**6 Exercise 6** **6**

**7 Exercise 7** **7**
- 7.1 Code . . . 7

**8 Exercise 8** **7**
- 8.1 Code trong while(1) . . . 7
- 8.2 Code spfftware timer . . . 8

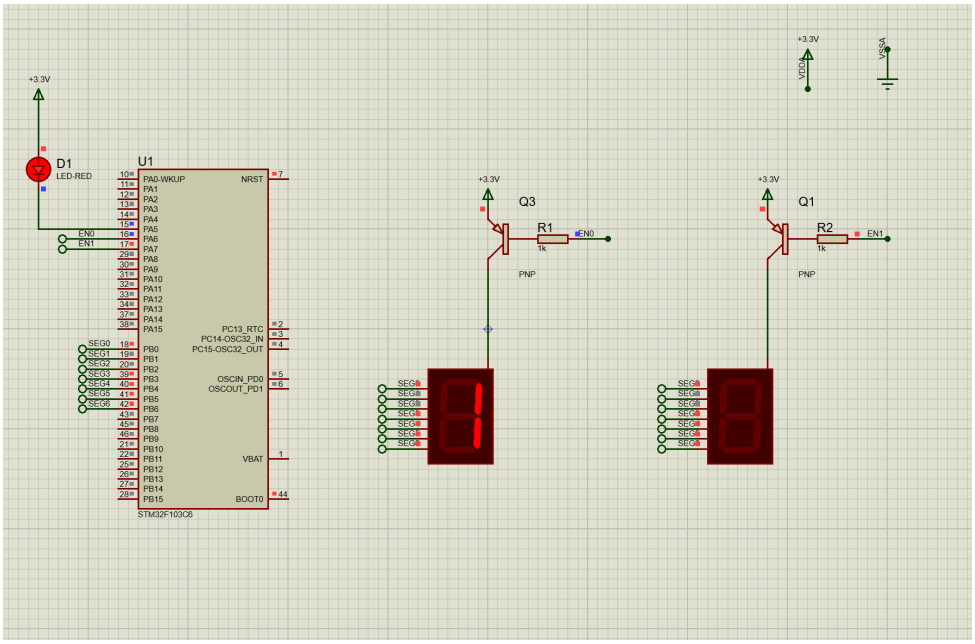
**9 Exercise 9** **9**
- 9.1 Sơ đồ Proteus . . . 9
- 9.2 Code . . . 9

**10 Exercise 10** **10**
- 10.1 Code function . . . 10
- 10.2 Code while(1) . . . 10

**11 Link github** **12**

# 1 Exercise 1

## 1.1 Sơ đồ Proteus




## 1.2 Code

```
 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    counter--;

    if (counter <= 0)
    {
        counter = 50;

        if (led_index == 0)
        {
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);

            display7SEG(1);

            led_index = 1;
        }
        else
        {
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);

            display7SEG(2);

            led_index = 0;
```

```
        }
    }
}
```

### 1.3 Short question

**What is the frequency of the scanning process?**

Hệ thống sử dụng 2 LED 7 đoạn. Theo yêu cầu, thời gian bật sáng cho mỗi LED (thời gian chuyển đổi - switching time) là $T_s = 0.5$ s (500 ms). Chu kỳ quét toàn bộ (tổng thời gian để bật sáng luân phiên hoàn tất cho cả 2 LED) được tính bằng:

$$T_{scan} = 2 \times T_s = 2 \times 0.5 \text{ s} = 1 \text{ s}$$

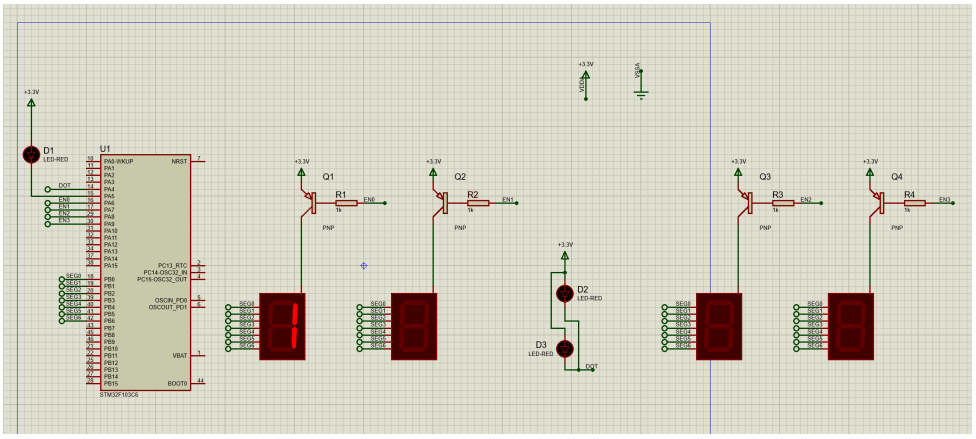
Tần số của quá trình quét (scanning frequency) là nghịch đảo của chu kỳ quét toàn bộ:

$$f_{scan} = \frac{1}{T_{scan}} = \frac{1}{1 \text{ s}} = 1 \text{ Hz}$$

**Kết luận:** Tần số của quá trình quét LED trong bài tập này là **1 Hz**.

## 2 Exercise 2

### 2.1 Sơ đồ Proteus



### 2.2 Code

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
dot_counter--;
if(dot_counter <= 0) {
  dot_counter = 100;
  HAL_GPIO_TogglePin(GPIOA, DOT_Pin);
}
led_counter --;
if (led_counter <= 0) {
  led_counter = 40;
     HAL_GPIO_WritePin(GPIOA, EN0_Pin | EN1_Pin | EN2_Pin |
 EN3_Pin, GPIO_PIN_SET);
```

```
11
12          display7SEG(led_buffer[led_index]);
13              if (led_index == 0) HAL_GPIO_WritePin(GPIOA, EN0_Pin,
    GPIO_PIN_RESET);
14              else if (led_index == 1) HAL_GPIO_WritePin(GPIOA,
    EN1_Pin, GPIO_PIN_RESET);
15              else if (led_index == 2) HAL_GPIO_WritePin(GPIOA,
    EN2_Pin, GPIO_PIN_RESET);
16              else if (led_index == 3) HAL_GPIO_WritePin(GPIOA,
    EN3_Pin, GPIO_PIN_RESET);
17              led_index++;
18              if (led_index == 4) led_index = 0;
19    }
20 }
```

### 2.3 Short question

**What is the frequency of the scanning process?**

**Answer:** Trong bài tập này, hệ thống đã được mở rộng lên 4 LED 7 đoạn. Thời gian hiển thị (switching time) cho mỗi LED vẫn được giữ nguyên là $T_s = 0.5$ s (500 ms). Vì có 4 LED 7 đoạn được bật luân phiên, chu kỳ quét toàn bộ (tổng thời gian để quét qua cả 4 LED) được tính bằng:

$$T_{scan} = 4 \times T_s = 4 \times 0.5 \text{ s} = 2 \text{ s}$$

Tần số của quá trình quét (scanning frequency) là nghịch đảo của chu kỳ quét toàn bộ:

$$f_{scan} = \frac{1}{T_{scan}} = \frac{1}{2 \text{ s}} = 0.5 \text{ Hz}$$

**Kết luận:** Tần số của quá trình quét LED trong hệ thống 4 LED này là **0.5 Hz**.

## 3 Exercise 3

### 3.1 Code

```
1  void update7SEG(int index) {
2    HAL_GPIO_WritePin(GPIOA, EN0_Pin | EN1_Pin | EN2_Pin | EN3_Pin,
     GPIO_PIN_SET);
3
4    display7SEG(led_buffer[led_index]);
5
6    switch(index) {
7      case 0: HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_RESET);
     break;
8      case 1: HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_RESET);
     break;
9      case 2: HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_RESET);
     break;
```

```
    case 3: HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_RESET);
   break;
  }
}
```

### 3.2 Code

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
  dot_counter--;
  if(dot_counter <= 0) {
    dot_counter = 100;
    HAL_GPIO_TogglePin(GPIOA, DOT_Pin);
  }
  led_counter --;
  if (led_counter <= 0) {
    led_counter = 50;
       HAL_GPIO_WritePin(GPIOA, EN0_Pin | EN1_Pin | EN2_Pin |
   EN3_Pin, GPIO_PIN_SET);


        update7SEG(led_index);
            led_index++;
            if (led_index == 4) led_index = 0;
  }
}
```

## 4 Exercise 4

### 4.1 Code

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
  dot_counter--;
  if(dot_counter <= 0) {
    dot_counter = 100;
    HAL_GPIO_TogglePin(GPIOA, DOT_Pin);
  }
  led_counter --;
  if (led_counter <= 0) {
    led_counter = 25;
       HAL_GPIO_WritePin(GPIOA, EN0_Pin | EN1_Pin | EN2_Pin |
   EN3_Pin, GPIO_PIN_SET);


        update7SEG(led_index);
            led_index++;
            if (led_index == 4) led_index = 0;
  }
}
```

# 5 Exercise 5

## 5.1 Code

```
void updateClockBuffer() {
  led_buffer[0] = hour / 10;
  led_buffer[1] = hour % 10;
  led_buffer[2] = minute / 10;
  led_buffer[3] = minute % 10;
}
```

# 6 Exercise 6

**Report 1: If in line 1 of the code above is miss, what happens after that and why? Answer:**

- **Hiện tượng:** Đèn LED sẽ **không bao giờ chớp tắt** (vĩnh viễn giữ nguyên trạng thái ban đầu).
- **Giải thích:** Khi khai báo, các biến `timer0_counter` và `timer0_flag` được khởi tạo bằng 0. Nếu thiếu dòng 1 (`setTimer0(1000);`), biến đếm `timer0_counter` sẽ không được gán giá trị khởi tạo. Do đó, hàm `timer_run()` (được gọi trong ngắt Timer mỗi 10ms) sẽ bỏ qua việc đếm lùi vì điều kiện `timer0_counter > 0` không thỏa mãn. Kéo theo đó, cờ `timer0_flag` sẽ không bao giờ được bật lên 1. Vì cờ không bật, điều kiện trong vòng lặp `while(1)` luôn sai, dẫn đến lệnh `TogglePin` không bao giờ được thực thi.

**Report 2: If in line 1 of the code above is changed to setTimer0(1), what happens after that and why? Answer:**

- **Hiện tượng:** Đèn LED vẫn **không bao giờ chớp tắt** tương tự như trường hợp 1.
- **Giải thích:** Hàm `setTimer0(int duration)` thực hiện phép tính:

$$\texttt{timer0_counter} = \frac{\texttt{duration}}{\texttt{TIMER_CYCLE}}$$

  Với `TIMER_CYCLE = 10`, nếu truyền `duration = 1`, ta có phép chia số nguyên 1/10. Trong ngôn ngữ C, phép chia giữa hai số nguyên sẽ bị làm tròn xuống (lấy phần nguyên), dẫn đến kết quả `timer0_counter = 0`. Vì biến đếm bằng 0 ngay từ đầu, cờ `timer0_flag` không bao giờ được bật, và hàm `TogglePin` sẽ không bao giờ được gọi.

**Report 3: If in line 1 of the code above is changed to setTimer0(10), what is changed compared to 2 first questions and why? Answer:**

- **Hiện tượng (Sự khác biệt):** Đèn LED **sẽ chớp tắt thành công**. Nó sẽ thay đổi trạng thái lần đầu tiên gần như ngay lập tức (sau đúng 10ms), và từ những lần sau trở đi, nó sẽ chớp tắt đều đặn với chu kỳ 2000ms.

- **Giải thích:** Khi gọi `setTimer0(10)`, phép tính trở thành $10/10 = 1$. Biến `timer0_counter` được gán giá trị hợp lệ là 1. Khi ngắt Timer xảy ra (10ms sau đó), hàm `timer_run()` sẽ trừ `counter` về 0 và kích hoạt cờ `timer0_flag = 1`. Vòng lặp `while(1)` bắt được cờ này, thực hiện lật trạng thái LED và gọi dòng `setTimer0(2000);` để bắt đầu chu kỳ đếm 2 giây mới. Do đó, hệ thống chớp tắt LED sử dụng Software Timer được khởi động thành công.

# 7 Exercise 7

## 7.1 Code

```
/* USER CODE BEGIN WHILE */
 while (1)
 {
   if (timer_flag[0] == 1) {
     setTimer(0, 1000);
     second++;
     if (second >= 60) { second = 0; minute++; }
     if (minute >= 60) { minute = 0; hour++; }
     if (hour >= 24) { hour = 0; }
     updateClockBuffer();
   }


   if (timer_flag[1] == 1) {
     setTimer(1, 500);
     HAL_GPIO_TogglePin(GPIOA, DOT_Pin);
   }


   /* USER CODE END WHILE */
     HAL_Delay(1000);
   /* USER CODE BEGIN 3 */
 }
 /* USER CODE END 3 */
```

# 8 Exercise 8

## 8.1 Code trong while(1)

```
while (1) {
   /* USER CODE END WHILE */

   /* USER CODE BEGIN 3 */

   if (timer_flag[0] == 1) {
     setTimer(0, 1000);
     second++;
```

```
      if (second >= 60) {
        second = 0;
        minute++;
      }
      if (minute >= 60) {
        minute = 0;
        hour++;
      }
      if (hour >= 24) {
        hour = 0;
      }
      updateClockBuffer();
    }

    if (timer_flag[1] == 1) {
      setTimer(1, 500);
      HAL_GPIO_TogglePin(GPIOA, DOT_Pin);
    }

    if (timer_flag[2] == 1) {
      setTimer(2, 10);
      update7SEG(led_index);

      led_index++;
      if (led_index >= MAX_LED) {
        led_index = 0;
      }
    }
  }
  /* USER CODE END 3 */
```

## 8.2 Code spfftware timer

```
 #define MAX_TIMERS 10
int timer_counter[MAX_TIMERS] = {0};
int timer_flag[MAX_TIMERS] = {0};
int TIMER_CYCLE = 10;

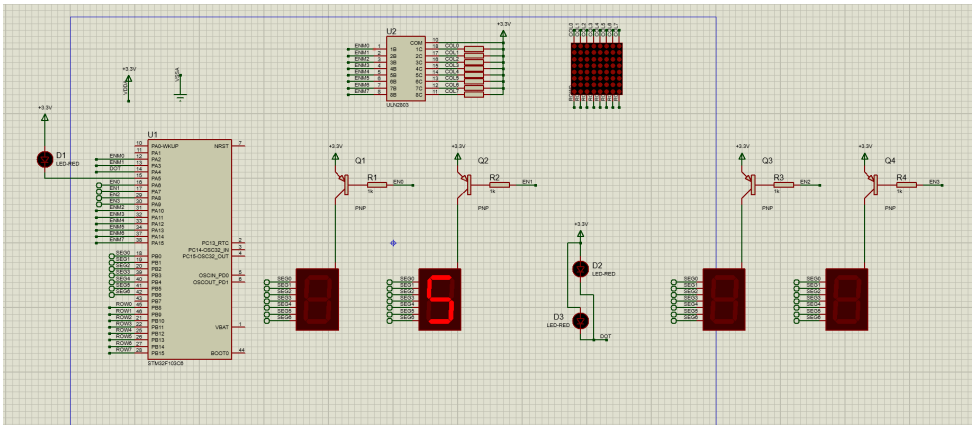
void setTimer(int index, int duration) {
  if (index >= 0 && index < MAX_TIMERS) {
    timer_counter[index] = duration / TIMER_CYCLE;
    timer_flag[index] = 0;
  }
}

void timerRun() {
  for (int i = 0; i < MAX_TIMERS; i++) {
    if (timer_counter[i] > 0) {
      timer_counter[i]--;
      if (timer_counter[i] <= 0) {
```

```
18          timer_flag[i] = 1;
19        }
20      }
21    }
22  }
```

# 9 Exercise 9

## 9.1 Sơ đồ Proteus



## 9.2 Code

```
1  const int MAX_LED_MATRIX = 8;
2  int index_led_matrix = 0;
3
4  uint8_t matrix_buffer[8] = {0x18, 0x3C, 0x66, 0x66, 0x7E, 0x7E, 0
       x66, 0x66};
5
6  void updateLEDMatrix(int index) {
7
8    HAL_GPIO_WritePin(GPIOA,
9                      GPIO_PIN_2 | GPIO_PIN_3 | GPIO_PIN_10 |
       GPIO_PIN_11 |
10                         GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 |
       GPIO_PIN_15,
11                     GPIO_PIN_RESET);
12
13   uint8_t row_data = matrix_buffer[index];
14
15   HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, (row_data >> 0) & 0x01);
16   HAL_GPIO_WritePin(GPIOB, GPIO_PIN_9, (row_data >> 1) & 0x01);
17   HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, (row_data >> 2) & 0x01);
18   HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, (row_data >> 3) & 0x01);
19   HAL_GPIO_WritePin(GPIOB, GPIO_PIN_12, (row_data >> 4) & 0x01);
20   HAL_GPIO_WritePin(GPIOB, GPIO_PIN_13, (row_data >> 5) & 0x01);
21   HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, (row_data >> 6) & 0x01);
```

```
  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_15, (row_data >> 7) & 0x01);

  switch (index) {
  case 0:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, GPIO_PIN_SET);
    break;
  case 1:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_SET);
    break;
  case 2:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET);
    break;
  case 3:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET);
    break;
  case 4:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_SET);
    break;
  case 5:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET);
    break;
  case 6:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET);
    break;
  case 7:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_SET);
    break;
  }
}
```

# 10 Exercise 10

## 10.1 Code function

```

void shiftLeftAnimation(void) {
  for (int i = 0; i < MAX_LED_MATRIX; i++) {
    uint8_t msb = (matrix_buffer[i] & 0x80) >> 7;
    matrix_buffer[i] = (matrix_buffer[i] << 1) | msb;
  }
}
```

## 10.2 Code while(1)

```

 while (1) {
    /* USER CODE END WHILE */

```

```
    /* USER CODE BEGIN 3 */

    if (timer_flag[0] == 1) {
      setTimer(0, 1000);
      second++;
      if (second >= 60) {
        second = 0;
        minute++;
      }
      if (minute >= 60) {
        minute = 0;
        hour++;
      }
      if (hour >= 24) {
        hour = 0;
      }
      updateClockBuffer();
    }

    if (timer_flag[1] == 1) {
      setTimer(1, 500);
      HAL_GPIO_TogglePin(GPIOA, DOT_Pin);
    }

    if (timer_flag[2] == 1) {
      setTimer(2, 250);
      update7SEG(led_index);

      led_index++;
      if (led_index >= MAX_LED) {
        led_index = 0;
      }
    }

    if (timer_flag[3] == 1) {
      setTimer(3, 10);
      updateLEDMatrix(index_led_matrix);

      index_led_matrix++;
      if (index_led_matrix >= MAX_LED_MATRIX) {
        index_led_matrix = 0;
      }
    }
  }

  if (timer_flag[4] == 1) {
     setTimer(4, 150);
     shiftLeftAnimation();
   }

  /* USER CODE END 3 */
```

# 11 Link github

Toàn bộ mã nguồn của các bài thực hành được lưu trữ trên GitHub tại địa chỉ:

- **Repository:** `https://github.com/haitran311/VXL_Lab_2_HCMUT`
- **Nội dung:** Mã nguồn STM32 và các tệp mô phỏng Proteus của các bài thực hành.